Characterizing Network Traffic with Programmable Data-Planes

Created	April 30, 2026
Created by	Efe Ertekin
Tags	Creative

Preparation & Setup

Since I haven't worked with P4/bmv2 before, I had to take a look at what they were about. As far as I understand, P4 is a language that lets you write your own packet-handling logic inside network switches/data planes. On the other hand, bmv2 is a software-based switch that lets you create a simulation to test your P4 codes before implementing them into the physical devices. [https://p4.org/, https://build-a-router-instructors.github.io/deliverables/p4-mininet/]

First, I downloaded the mentioned PCAP file and then dumped the first 10,000 packets to make sure that I have enough packets, and also, this is the minimum number of packets that needs to be used for the final results. Here is the code that I used to get the first 10,000 packets:

```
tcpdump -r 201302011400.dump -c 10000 -w subset_10k.pcap
```

Not relevant to the report, but I was abroad while working on this project. So, I only had my MacBook Air M1 with me. To keep things move faster, I decided to move forward with Google Colab for Task 1. I uploaded ***subset_10k.pcap*** to my Google Drive so that I can reach it through Google Colab.

Before moving to the Google Colab part, I downloaded the VM that has P4 development tools inside of it using this link: https://github.com/jafingerhut/p4-guide/blob/master/bin/README-install-troubleshooting.md. Specifically, this one: "2026-Apr-01" - "aarch64 VM Image link". And then checked whether it works fine

or not. Since this is my first time working with these tools, I asked **Claude Sonnet 4.6** how to check whether my setup is ready or not. It helped me to check P4 Compiler, bmv2 Simple Switch, simple_switch_grpc, and Python gRPC libraries. I had an issue with p4runtime_lib, but I fixed it with the help of **Claude Sonnet 4.6**. Apparently, p4runtime_lib is not a standard pip package, and it lives inside the tutorials repo. We had to fix that issue. Here are the commands that I used:

Check P4 Compiler

`p4c --version`

Check bmv2 Simple Switch

`simple_switch --version`

Check simple_switch_grpc

`which simple_switch_grpc`

`simple_switch_grpc --version`

Check Python gRPC libraries

`python3 -c "import grpc; print('gRPC OK:', grpc.__version__)"`

`python3 -c "import p4runtime_lib; print('p4runtime OK')"`

Fix the p4runtime_lib issue:

`echo 'export PYTHONPATH=$PYTHONPATH:~/tutorials/utils' >> ~/.bashrc`

`source ~/.bashrc`

Now the setup is ready to start working on this homework.

The Usage of LLMs

Before moving on to the next steps, the code inside this project was mostly (at least 90%) written with **Claude Sonnet 4.6** through its website. As I have mentioned, I haven't used P4 before, and this was my first time. To be honest, even if I knew how to code with it, I'd still rely on LLMs to build things faster. In that case, I'd have more control over the code rather than trusting the code by using different methods. Here are the methods that I used to verify the written code:

Asked Claude these:

1. "Explain this code."

2. “Convert to pseudo-code.”
3. “List bugs & edge cases.”

With the second question, you can easily capture the essence of the written code. In addition to these, in some parts, I asked **Gemini 3 (Thinking)** to cross-check whether some code is correct or not.

I also asked **Claude Sonnet 4.6** to come up with proper mathematical models that might fit this data. I’m not fully familiar with all of those distributions.

Learning the fundamentals of the languages and frameworks I use, and then relying on LLMs, would be a better way to handle things.

Task I. Software-based Traffic Characterization and Statistical Analysis

1. First, I loaded the PCAP file ***subset_10k.pcap*** that I created before and uploaded to Google Drive.
2. And then I extracted these features: timestamp (arrival time), size (packet in bytes), iat (inter-arrival time), proto (protocol type), src_ip, dst_ip, src_port, dst_port.
3. Both packet size and IAT distribution were shown here. (*Probability Density Function*)
4. Performing *Statistical Distribution Fitting* to see which mathematical model describes the IAT of the network traffic. The fitting part calculates the right parameters for each model and creates its best version for this traffic. *The KS statistics* part is used to calculate the vertical distance between the distribution of the data and the theoretical model. Lower value means better fit. *The Akaike Information Criterion (AIC)* is a scoring system that rewards good fit but penalizes models that are overly complex. The lowest is generally considered the best.
5. Visual comparison of best-fitted and worst-fitted theoretical models on data in the IAT. Pareto ends up the best one.
6. Visualization of TCP and UDP packets in terms of size and IAT.

7. The analysis of short and long flows. We can look at this as mouse flows and elephant flows as well.
8. I wrote a code snippet to see whether the number of samples used changes the p-value. And it does. With the large amount of data, the p-value starts rejecting everything and becomes useless to rank theoretical models.
9. Skewness is higher than 1 with 2.4, which means the average IAT is basically a lie. Because most IAT values are really close to 0. Instead of relying on the mean, taking the consideration of the median is a better way to understand typical delay in this network.
10. Even in random sampling for n=20, Pareto is still the best.

Virtual Machine (VM) Setup Part

Step 1. Set up your working folder in the VM

`mkdir ~/traffic_homework`

`cd ~/traffic_homework`

Step 2. Copy your PCAP into the VM

I moved ***subset_10k.pcap*** to the shared folder. So, I can reach that through my VM and Mac. I used VirtualBox as the VM program.

Step 3. Write the P4 program

`cd ~/traffic_homework`

`nano traffic.p4`

```
/* traffic.p4 - Packet feature extraction for traffic charact
erization */

#include <core.p4>
#include <v1model.p4>

/* ===== CONSTANTS ===== */
#define MAX_FLOWS 4096
#define FLOW_HASH_WIDTH 12  // 2^12 = 4096 buckets
```

```
/* ===== HEADERS ===== */
header ethernet_t {
    bit<48> dst_addr;
    bit<48> src_addr;
    bit<16> ether_type;
}

header ipv4_t {
    bit<4>  version;
    bit<4>  ihl;
    bit<8>  diffserv;
    bit<16> total_len;
    bit<16> identification;
    bit<3>  flags;
    bit<13> frag_offset;
    bit<8>  ttl;
    bit<8>  protocol;
    bit<16> hdr_checksum;
    bit<32> src_addr;
    bit<32> dst_addr;
}

header tcp_t {
    bit<16> src_port;
    bit<16> dst_port;
    bit<32> seq_no;
    bit<32> ack_no;
    bit<4>  data_offset;
    bit<4>  res;
    bit<8>  flags;
    bit<16> window;
    bit<16> checksum;
    bit<16> urgent_ptr;
}
```

```
header udp_t {
    bit<16> src_port;
    bit<16> dst_port;
    bit<16> length;
    bit<16> checksum;
}

/* Custom header to carry computed IAT */
header custom_t {
    bit<32> iat_us;   // IAT in microseconds
    bit<32> flow_id;  // Hash-based flow ID
}

struct headers_t {
    ethernet_t ethernet;
    ipv4_t     ipv4;
    tcp_t      tcp;
    udp_t      udp;
    custom_t   custom;
}

struct metadata_t {
    bit<32> flow_id;
    bit<32> iat_us;
}

/* ===== PARSER ===== */
parser MyParser(packet_in pkt,
                out headers_t hdr,
                inout metadata_t meta,
                inout standard_metadata_t std_meta) {

    state start {
        transition parse_ethernet;
    }
```

```
    state parse_ethernet {
        pkt.extract(hdr.ethernet);
        transition select(hdr.ethernet.ether_type) {
            0x0800: parse_ipv4;
            default: accept;
        }
    }

    state parse_ipv4 {
        pkt.extract(hdr.ipv4);
        transition select(hdr.ipv4.protocol) {
            6:  parse_tcp;
            17: parse_udp;
            default: accept;
        }
    }

    state parse_tcp {
        pkt.extract(hdr.tcp);
        transition accept;
    }

    state parse_udp {
        pkt.extract(hdr.udp);
        transition accept;
    }
}

/* ===== CHECKSUM VERIFICATION ===== */
control MyVerifyChecksum(inout headers_t hdr, inout metadata_
t meta) {
    apply { }
}

/* ===== INGRESS PROCESSING ===== */
control MyIngress(inout headers_t hdr,
```

```
                    inout metadata_t meta,
                    inout standard_metadata_t std_meta) {

    /* Register to store last packet timestamp (global IAT)
*/
    register<bit<32>>(1) last_timestamp_reg;
    register<bit<32>>(10000) iat_samples;
    register<bit<32>>(1)     iat_index;

    /* Registers for flow-level stats */
    register<bit<32>>(MAX_FLOWS) flow_packet_count;
    register<bit<32>>(MAX_FLOWS) flow_byte_count;
    register<bit<32>>(MAX_FLOWS) flow_last_time;

    action compute_flow_id() {
        bit<32> flow_hash;
        hash(flow_hash,
             HashAlgorithm.crc32,
             (bit<32>)0,
             { hdr.ipv4.src_addr,
               hdr.ipv4.dst_addr,
               hdr.ipv4.protocol },
             (bit<32>)MAX_FLOWS);
        meta.flow_id = flow_hash;
    }

    action compute_iat() {
        bit<32> last_time;
        bit<32> current_time;
        bit<32> idx;
        current_time = (bit<32>)std_meta.ingress_global_times
tamp;
        last_timestamp_reg.read(last_time, 0);
        meta.iat_us = current_time - last_time;
        last_timestamp_reg.write(0, current_time);
```

```
        // Store IAT sample
        iat_index.read(idx, 0);
        iat_samples.write(idx, meta.iat_us);
        iat_index.write(0, idx + 1);
     }

    action update_flow_stats() {
        bit<32> pkt_count;
        bit<32> byte_count;
        flow_packet_count.read(pkt_count, meta.flow_id);
        flow_byte_count.read(byte_count, meta.flow_id);
        pkt_count = pkt_count + 1;
        byte_count = byte_count + (bit<32>)std_meta.packet_le
ngth;
        flow_packet_count.write(meta.flow_id, pkt_count);
        flow_byte_count.write(meta.flow_id, byte_count);
    }

    action add_custom_header() {
        hdr.custom.setValid();
        hdr.custom.iat_us  = meta.iat_us;
        hdr.custom.flow_id = meta.flow_id;
    }

    action drop() {
        mark_to_drop(std_meta);
    }

    table ipv4_forward {
        key = {
            hdr.ipv4.dst_addr: lpm;
        }
        actions = {
            drop;
            NoAction;
        }
```

```
        default_action = NoAction();
    }

    apply {
        if (hdr.ipv4.isValid()) {
            compute_flow_id();
            compute_iat();
            update_flow_stats();
            add_custom_header();
            ipv4_forward.apply();
        }
    }
}

/* ===== EGRESS PROCESSING ===== */
control MyEgress(inout headers_t hdr,
                 inout metadata_t meta,
                 inout standard_metadata_t std_meta) {
    apply { }
}

/* ===== CHECKSUM COMPUTATION ===== */
control MyComputeChecksum(inout headers_t hdr, inout metadata
_t meta) {
    apply { }
}

/* ===== DEPARSER ===== */
control MyDeparser(packet_out pkt, in headers_t hdr) {
    apply {
        pkt.emit(hdr.ethernet);
        pkt.emit(hdr.ipv4);
        pkt.emit(hdr.tcp);
        pkt.emit(hdr.udp);
        pkt.emit(hdr.custom);
    }
```

```
}

/* ===== SWITCH INSTANTIATION ===== */
V1Switch(
    MyParser(),
    MyVerifyChecksum(),
    MyIngress(),
    MyEgress(),
    MyComputeChecksum(),
    MyDeparser()
) main;
```

Step 4. Compile it with p4c & Create a Virtual Network Interface

```
p4c --target bmv2 --arch v1model \
--std p4-16 \
--p4runtime-files traffic.p4info.txt \
traffic.p4
```

After the execution of the command above, you'll see a ***traffic.json*** file.

```
# Create a virtual ethernet pair
sudo ip link add veth0 type veth peer name veth1

# Bring both interfaces up
sudo ip link set veth0 up
sudo ip link set veth1 up

# Disable offloading (important for bmv2)
sudo ethtool -K veth0 rx off tx off
sudo ethtool -K veth1 rx off tx off

# Verify they exist
ip link show veth0
ip link show veth1
```

Step 5. Start simple_switch_grpc with thrift port

bmv2 has a built-in thrift CLI that fully supports register reads. Even though it's mentioned that gRPC is easier in this case, I decided to move on with CLI because I came across a problem like:

> grpc._channel._MultiThreadedRendezvous: <_MultiThreadedRendezvous of RPC that terminated with:
> status = StatusCode.UNIMPLEMENTED
> details = "Register reads are not supported yet"
> debug_error_string = "UNKNOWN:Error received from peer ipv4:127.0.0.1:50051 {grpc_message:"Register reads are not supported yet", grpc_status:12, created_time:"2026-04-30T20:27:00.859161421+00:00"}"

> The error message is clear: `simple_switch_grpc` doesn't support reading registers via P4Runtime (`"Register reads are not supported yet"`). This is a known BMv2 limitation — the gRPC server implements P4Runtime but register reads via that interface are not implemented.

```
sudo simple_switch_grpc \
--device-id 0 \
-i 0@veth0 \
--log-console \
--thrift-port 9090 \
traffic.json \
-- \
--grpc-server-addr 0.0.0.0:50051
```

Step 6. Write the Python gRPC controller

Open a new terminal.

```
cd ~/traffic_homework
```

```
nano controller.py
```

```
#!/usr/bin/env python3
"""
controller.py - Reads flow stats from simple_switch_grpc via
simple_switch_CLI
                (P4Runtime gRPC used only for pipeline config;
register reads
                 go through the Thrift CLI because BMv2 gRPC d
```

```
oes not support them)
"""
import csv
import time
import subprocess
import p4runtime_lib.bmv2
import p4runtime_lib.helper
from p4runtime_lib.switch import ShutdownAllSwitchConnections

# ===== CONFIG =====
P4INFO_FILE  = 'traffic.p4info.txt'
BMV2_JSON    = 'traffic.json'
SWITCH_ADDR  = '127.0.0.1:50051'
DEVICE_ID    = 0
THRIFT_PORT  = 9090          # adjust if you started bmv2 wit
h --thrift-port XXXX
MAX_FLOWS        = 4096
MAX_IAT_SAMPLES  = 10000


# ------------------------------------------------------------
----------------
# Switch connection (gRPC / P4Runtime — used only for pipelin
e config)
# ------------------------------------------------------------
----------------

def connect_to_switch():
    """Connect via P4Runtime gRPC and push the forwarding pip
eline."""
    p4info_helper = p4runtime_lib.helper.P4InfoHelper(P4INFO_
FILE)
    sw = p4runtime_lib.bmv2.Bmv2SwitchConnection(
        name='traffic-sw',
        address=SWITCH_ADDR,
        device_id=DEVICE_ID,
```

```
        proto_dump_file='p4runtime_requests.txt'
    )
    sw.MasterArbitrationUpdate()
    sw.SetForwardingPipelineConfig(
        p4info=p4info_helper.p4info,
        bmv2_json_file_path=BMV2_JSON
    )
    print("✓ Connected to switch and pipeline configured")
    return sw, p4info_helper


# ------------------------------------------------------------
----------------
# Register reads via simple_switch_CLI (Thrift)
# ------------------------------------------------------------
----------------

def read_register_index(register_name: str, index: int,
                        thrift_port: int = THRIFT_PORT) -> in
t:
    """
    Read a single register slot via simple_switch_CLI.
    Output format from BMv2:  'MyIngress.flow_packet_count[4
2]= 17\nRuntimeCmd: '
    We find the line containing the register name, split on
'=', and take
    the first whitespace token to discard the trailing 'Runti
meCmd:' prompt.
    """
    cmd = (f"echo 'register_read {register_name} {index}' | "
           f"simple_switch_CLI --thrift-port {thrift_port}")
    out = subprocess.check_output(cmd, shell=True,
                                  stderr=subprocess.DEVNULL).
decode()
    for line in out.split('\n'):
        if register_name in line and '=' in line:
```

```
            return int(line.split('=')[-1].strip().split()
[0])
    return 0


def read_all_flows(thrift_port: int = THRIFT_PORT) -> list:
    """
    Scan all MAX_FLOWS slots.  For each slot we first read th
e packet counter;
    only if it is non-zero do we pay the cost of a second CLI
call for bytes.
    This mirrors the working bash approach but lives inside c
ontroller.py.
    """
    print("\tReading registers from switch (this may take a m
oment)...")
    flows = []

    for i in range(MAX_FLOWS):
        pkt_count = read_register_index('MyIngress.flow_packe
t_count', i,
                                        thrift_port)
        if pkt_count > 0:
            byte_count = read_register_index('MyIngress.flow_
byte_count', i,
                                             thrift_port)
            flows.append((i, pkt_count, byte_count))

        if i % 512 == 0:
            print(f"\t  Scanned {i}/{MAX_FLOWS} flow slot
s...")

    return flows


# ------------------------------------------------------------
```

```
----------------
# IAT samples
# ------------------------------------------------------------
----------------

def read_iat_samples(thrift_port: int = THRIFT_PORT) -> list:
    """
    Read inter-arrival time samples from the switch.
    First reads iat_index[0] to find how many samples were st
ored,
    then reads that many entries from iat_samples[].
    Returns a list of non-zero IAT values in microseconds.
    """
    print("\tReading IAT samples from switch...")

    iat_count = read_register_index('MyIngress.iat_index', 0,
thrift_port)
    print(f"\tTotal IAT samples stored: {iat_count}")

    iat_values = []
    limit = min(iat_count, MAX_IAT_SAMPLES)

    for i in range(limit):
        val = read_register_index('MyIngress.iat_samples', i,
thrift_port)
        if val > 0:
            iat_values.append(val)
        if i % 1000 == 0 and i > 0:
            print(f"\t  Read {i}/{limit} samples...")

    return iat_values


# ------------------------------------------------------------
----------------
# Main
```

```
# ------------------------------------------------------------
----------------

def main():
    sw, _ = connect_to_switch()

    print("\nWaiting for traffic... (will read registers afte
r 30 seconds)")
    print("Run tcpreplay in another terminal now!")
    time.sleep(30)

    flows = read_all_flows(THRIFT_PORT)
    iat_values = read_iat_samples(THRIFT_PORT)

    # ------ Report ------
    print(f"\n{'='*60}")
    print(f"{'Flow ID':<12}{'Packets':<15}{'Bytes':<15}{'Avg
Pkt Size':<15}")
    print(f"{'='*60}")

    total_packets = 0
    total_bytes   = 0

    for flow_id, pkt_count, byte_count in sorted(flows, key=l
ambda x: x[1],
                                                 reverse=Tru
e):
        avg_size = byte_count / pkt_count if pkt_count > 0 el
se 0
        print(f"{flow_id:<12}{pkt_count:<15}{byte_count:<15}
{avg_size:<15.1f}")
        total_packets += pkt_count
        total_bytes   += byte_count

    print(f"{'='*60}")
    print(f"Total active flows : {len(flows)}")
```

```
    print(f"Total packets      : {total_packets}")
    print(f"Total bytes        : {total_bytes}")

    # ------ Task II Summary ------
    if flows:
        avg_sizes = [b / p for _, p, b in flows]
        print(f"\nTask II (P4) Results:")
        print(f"  Active flows      : {len(flows)}")
        print(f"  Total packets     : {total_packets}")
        print(f"  Avg packet size   : {sum(avg_sizes)/len(avg
_sizes):.1f} bytes")
        print(f"  Min avg pkt size  : {min(avg_sizes):.1f} by
tes")
        print(f"  Max avg pkt size  : {max(avg_sizes):.1f} by
tes")

    # ------ Save CSV ------
    with open('flow_stats.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['flow_id', 'packet_count', 'byte_cou
nt', 'avg_pkt_size'])
        for flow_id, pkt_count, byte_count in flows:
            avg_size = byte_count / pkt_count if pkt_count >
0 else 0
            writer.writerow([flow_id, pkt_count, byte_count,
round(avg_size, 2)])

    print("\n✓ Results saved to flow_stats.csv")

    # ------ IAT Report ------
    if iat_values:
        print(f"\n{'='*60}")
        print(f"IAT Samples Summary")
        print(f"{'='*60}")
        print(f"Non-zero samples : {len(iat_values)}")
        print(f"Avg IAT          : {sum(iat_values)/len(iat_v
```

```
alues):.2f} us")
        print(f"Min IAT          : {min(iat_values)} us")
        print(f"Max IAT          : {max(iat_values)} us")
        print(f"{'='*60}")

        with open('iat_samples.csv', 'w', newline='') as f:
            writer = csv.writer(f)
            writer.writerow(['iat_us'])
            for v in iat_values:
                writer.writerow([v])
        print("✓ IAT samples saved to iat_samples.csv")
    else:
        print("\n⚠ No IAT samples found.")

    ShutdownAllSwitchConnections()


if __name__ == '__main__':
    main()
```

`python3 controller.py`

The results will be saved to ***flow_stats.csv*** & ***iat_samples.csv***.

Step 7. Replay the PCAP

Open a new terminal.

`cd ~/traffic_homework`

`sudo tcpreplay -i veth1 --multiplier=0.01 /media/sf_Shared_Folder/subset_10k.pcap`

Task II-III. Comparison and Discussion

1. Uploaded ***flow_stats.csv*** and ***iat_samples.csv*** that are extracted from P4 to Google Drive, and then loaded them to Google Colab to work on it.
2. Comparison of Scapy and P4 is shown here. Total packet numbers are exactly the same as expected. The difference of average packet sizes in bytes is just below 10%, which is okay for this case. However, the active flows' difference

is a bit high with 19.1%. The reason for this is most probably hash collisions. Flow identification relies on a CRC32 hash into 4,096 buckets using only src_ip, dst_ip, and protocol, without port numbers. Multiple distinct flows can map to the same bucket and lead to lower active flow numbers.

3. For the IAT comparison of Task I and II, the means, median, and standard deviations are pretty much identical, which means our P4/bmv2 simulation is pretty much identical to Scapy.
4. Min and max values are a bit off, and I believe that the reason behind this is when tcpreplay sends two packets in rapid sequence, bmv2 may process them with a very small/large internal gap that doesn't correspond to the real IAT in the original trace.
5. In terms of overhead, Task I (Python/Scapy) is computationally expensive due to the fact that the whole PCAP file needs to fit inside the memory before doing the calculations. CPU usage also scales with the traffic volume. On the other hand, in Task II (P4/bmv2), feature extraction is happening in the data plane. I mean, I'm assuming that we're running this whole system in real hardware rather than a simulation like this. In that case, switches are super fast.
6. Task I doesn't scale properly. Memory usage increases as the trace increases. We also need captured data to be able to process it. However, in Task II, processing per packet is constant and does not rely on the traffic volume.
7. Comparison of ease of implementation, Task I is way easier than Task II because it requires mostly known libraries and languages. Unfortunately, Task II has a more complex environment setup as well as specialized language for it. Basically, it has a steep learning curve.